



UNIVERSITEIT VAN PRETORIA  
UNIVERSITY OF PRETORIA  
YUNIBESITHI YA PRETORIA

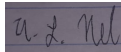
Faculty of Engineering, Built Environment and  
Information Technology

## EDC 310

### DIGITAL COMMUNICATIONS

#### PRACTICAL 2: REPORT

#### GROUP 8

| Name and Surname | Student Number | Signature                                                                            | % Contribution |
|------------------|----------------|--------------------------------------------------------------------------------------|----------------|
| A. van Niekerk   | 19007583       |  | 50             |
| A.S. Nel         | 19014432       |  | 50             |

By signing this assignment we confirm that we have read and are aware of the University of Pretoria's policy on academic dishonesty and plagiarism and we declare that the work submitted in this assignment is our own as delimited by the mentioned policies. We explicitly declare that no parts of this assignment have been copied from current or previous students' work or any other sources (including the internet), whether copyrighted or not. We understand that we will be subjected to disciplinary actions should it be found that the work we submit here does not comply with the said policies.

November 15, 2021

# Contents

|                          |                                         |           |
|--------------------------|-----------------------------------------|-----------|
| 1                        | Introduction and Aim . . . . .          | 2         |
| 2                        | DFE . . . . .                           | 3         |
| 3                        | MLSE . . . . .                          | 4         |
| 4                        | Simulation . . . . .                    | 5         |
|                          | 4.1 Design and Implementation . . . . . | 5         |
| 5                        | Results . . . . .                       | 7         |
| 6                        | Discussion . . . . .                    | 8         |
| 7                        | Conclusion . . . . .                    | 9         |
| <b>A DFE Code</b>        |                                         | <b>11</b> |
| <b>B MLSE Code</b>       |                                         | <b>12</b> |
| <b>C Simulation Code</b> |                                         | <b>17</b> |
| <b>D Question 3</b>      |                                         | <b>21</b> |
| <b>E Question 1</b>      |                                         | <b>28</b> |

# List of Figures

|                                                                                          |   |
|------------------------------------------------------------------------------------------|---|
| 1.0.1 BPSK Modulation . . . . .                                                          | 2 |
| 1.0.2 4QAM Modulation . . . . .                                                          | 2 |
| 1.0.3 8PSK Modulation . . . . .                                                          | 3 |
| 3.0.1 Example of a trellis for the BPSK modulation scheme . . . . .                      | 4 |
| 5.0.1 BER as a function of SNR for a BPSK with DFE and MLSE avareged over 100 iterations | 7 |
| 5.0.2 BER as a function of SNR for a 4QAM with DFE and MLSE avareged over 100 iterations | 7 |
| 5.0.3 BER as a function of SNR for a 8PSK with DFE and MLSE avareged over 100 iterations | 8 |

# 1 Introduction and Aim

The aim of practical 2 is to design and implement a simulation platform for a communication system transmitting information over an additive white Gaussian noise (AWGN) multi-path channel using Decision Feedback Equalisation (DFE) and Maximum Likelihood Sequence Estimation (MLSE). The simulation platform that was developed for practical 1 will be used as the foundation.

Whenever a signal is transmitted, it will most likely be distorted for a number of reasons such as the medium and length of transmission. A multipath channel may distort the signal through noise which causes the a misrepresentation of the transmitted signal on the receiver's end. The main reason multipath channels may cause signal distortion is because of intersymbol interference which means that one transmitted symbol can cause interference with subsequently transmitted symbols.

Noise refers to unintended (and, in most cases, unforeseen) changes to a signal that occur during capture, storage, transmission, processing, or conversion. Additive noise refers to noise that is added to the transmitted signal. AWGN is a model that is used to replicate the effect of noise that would usually occur in nature and practical implementation of signal transmission and processing. The the AWGN simulation of practical 1 will be used for practical 2.

Because there are many electromagnetic routes leading to the receiver, several instances of the same signal may be received at various times, creating signal distortion at the receiver's end. A channel impulse response (CIR) model is created in order to simulate the effects of multipath and determine the intended signal at the receiver. The CIR depicts how the signal varies in frequency and time inside the channel.

For practical 2, simulations should be run for both static and dynamic CIRs. A static CIR means that the CIR stays the same for all transmitted data, whereas a dynamic CIR means a new CIR is generated for each transmitted data block.

The effects of using DFE and MLSE will be investigated. The following modulation constellations will be used for each: BPSK, 4QAM and 8PSK.

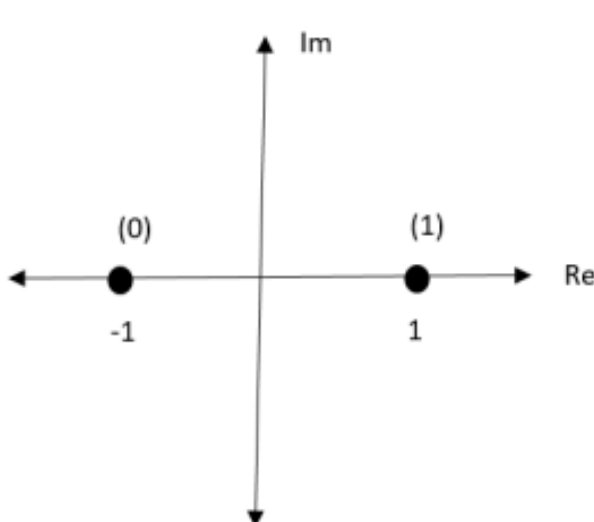


Figure 1.0.1: BPSK Modulation

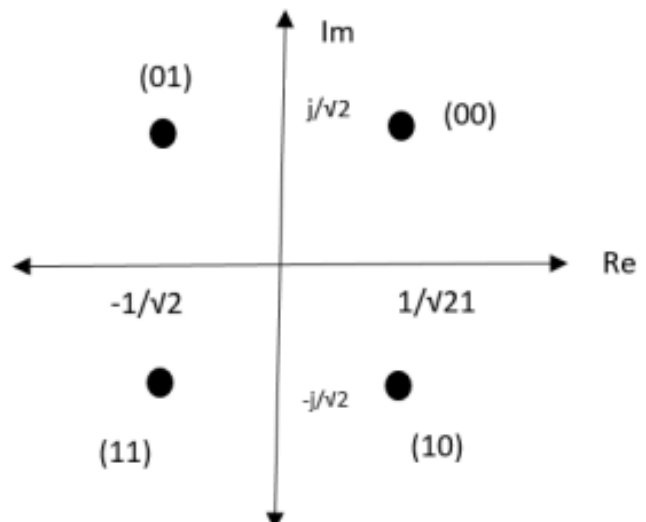


Figure 1.0.2: 4QAM Modulation

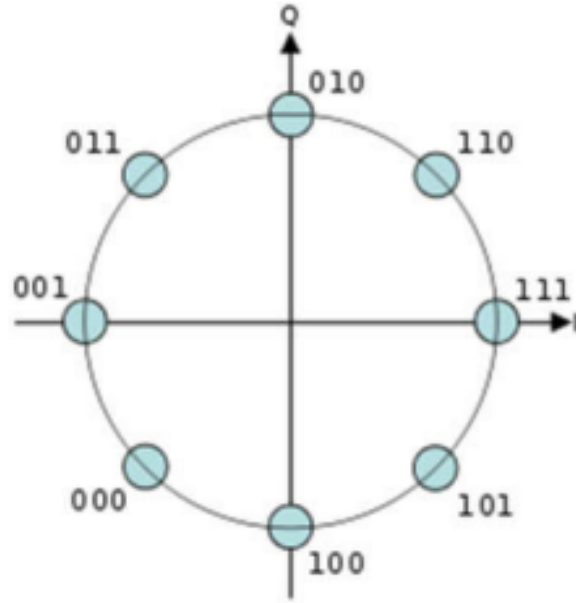


Figure 1.0.3: 8PSK Modulation

By the end of the simulation, plots of the bit error rate (BER) vs the signal-to-noise ratio (SNR) must be generated for both DFE and MLSE and for each of the aforementioned modulation schemes.

## 2 DFE

The Decision Feedback Equalizer (DFE) algorithm is a filter that reduces Inter Signal Interference (ISI) and detects transmitted signals by combining previously recognized symbols with a linear equalizer. The header symbols, data symbols, and tail symbols are all part of a block of symbols broadcast from a transmitting antenna to a receiving antenna. Before transmission, the header and tail symbols are prepended and appended to the data symbols, and assumes the first value of the current constellation used, this number of appended and prepended header and tail symbols equals the number of Channel Impulse Response elements minus one. However, in general, the DFE algorithm does not make a difference.

The DFE algorithm is identical to the MLSE algorithm, in the sense that both equalizers compute the costs of deltas for each state by:

$$\Delta = |\text{Received\_Symbol} - s_k C_0 + s_{k-1} C_1 \dots + s_{k-n} C_n|^2 \quad (1.1)$$

However, DFE differs from MLSE in that it only considers the cost from one node to the next when calculating the symbol received, rather than the total path cost, and so is more vulnerable to the repercussions of symbol estimation errors.

The DFE algorithm is implemented creating a DFE function to compute and return the identified symbols. As constructor parameters, the function uses the received bit sequence, the Channel Impulse Response vector, the length of the channel impulse response and the chosen constellation. The DFE algorithm computes the delta values for each conceivable state for each bit received, using equation 1.1 above.

The symbol that results in the least delta value for each time instant is the chosen symbol at that specific time instant. Before putting the result in its member variable, the algorithm estimates all

symbols received. The user can then use the DFE() function, this function retrieves the detected symbols.

### 3 MLSE

To discover the most likely sequence of symbols received at a receiving transmitter, the Maximum Likelihood Sequence Estimation (MLSE) technique is utilized. The header symbols, data symbols, and tail symbols are all part of a block of symbols broadcast from a transmitting antenna to a receiving antenna.

The Maximum Likelihood Sequence Estimation (MLSE) algorithm makes use of a modified decision tree called, a trellis, and an optimal search strategy to optimally detect sequences of transmitted symbols. This algorithm is known as the Viterbi algorithm and traverses the decision tree in search of the shortest path, which corresponds to the highest probability and thus the optimal estimated sequence.

The transition costs for moving from one trellis node to another is calculated with:

$$\Delta_n^{sk} = |s_k C_0 + s_{k-1} C_1 + \dots + s_{k-n} C_n|^2 \quad (1.2)$$

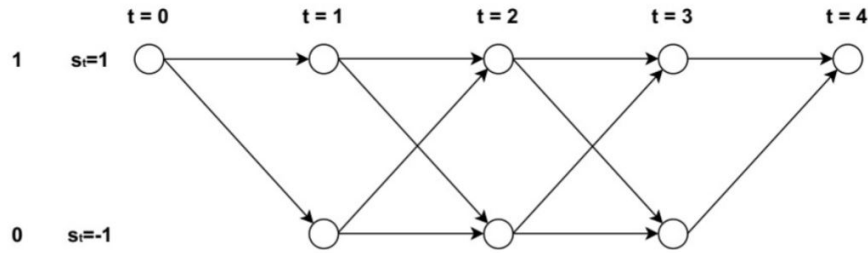


Figure 3.0.1: Example of a trellis for the BPSK modulation scheme

The goal of the trellis is to eliminate the time used for repetitive calculations along sub-optimal paths. By pruning a sub-optimal path, all further calculations that would have been necessary to calculate along future branching paths are eliminated as well, saving memory as well as processing power.

When a sequence of symbols is transmitted, a few known symbols are appended to the front and the back of the sequence, known as headers and tails in order to start off the cost calculations of state transitions. All transition state costs from one symbol to another are calculated for each symbol in the sequence, and the cheapest cost is retained and stored in the previous node along the path.

When the all costs along all of the cheapest paths are calculated, the final, absolute shortest path is maintained and the final sequence is found by removing header and tail symbols.

At the start of running the MLSE function, a number of bits are generated using the uniform random number generator from practical 1 and passing these numbers to another function from practical one to establish if a bit should be 1 or 0. Next, bits are mapped to symbols, depending on the modulation scheme that is selected.

when bits are transmitted, noise is added to the bit sequence through function that takes the sequence, CIR, modulation scheme, length of the CIR and SNR value as input parameters to determine the nature of the added noise.

A function has been implemented to enable the effects of a communication channel with memory, which also takes a specific modulation scheme as an input parameter and the length of memory to generate the possible transition states.

Three separate functions have been implemented to calculate the transition cost,  $\Delta$ , between two trellis nodes for each modulation scheme. This is followed by three functions that implement the actual MLSE algorithm and makes use of all previous functions to execute the algorithm.

Finally, two functions, both taking any modulation scheme as an input parameter as well a number of iterations, execute the MLSE algorithm. One function executes the algorithm for a static CIR and another for a dynamic CIR. These functions return the average bit error rate which can be used to plot the results in the main simulation file.

## 4 Simulation

### 4.1 Design and Implementation

The BPSK,4QAM and 8PSK simulation platform that was developed for practical 1 was used to simulate the effect multipath has on the received signal. For this practical two channel impulse responses was used, both is lengths of 3, The first channel impulse response being static and does not change and the second one being dynamic and changes every time a new response is generated. The simulation platform would simulate the average BER vs SNR of all modulation types using 200 data bits. Increasing the average sampling rate improves graph quaiaty but greatly increasing computation time, this can be done by increasing the numIterations at the bottom of the sumilation code.

The platform was generated using the Simulation class. As default parameters the class takes in the channel impulse response if not generated randomly as an array called `linC`, the number of data bits transmitted and the number of iterations that the BER should be avraged over. The class also consist of a number of member functions to perform the simulation of MLSE and DFE for BPSK, 4QAM and 8PSK.

The randomly generated channel impulse response was generated using equation 1.3 bellow. Where  $\varkappa$  is a value generated our random Gauss generator from our practical 1. The dynamic channel input respone is generatred by the `generateRandomCIR()` function part of the simulation class.

$$\mathbf{c}_{dynamic} = \left\{ \frac{\varkappa + j\varkappa}{\sqrt{2.3}}, \frac{\varkappa + j\varkappa}{\sqrt{2.3}}, \frac{\varkappa + j\varkappa}{\sqrt{2.3}} \right\} \quad (1.3)$$

The Simulation functions as follows. Firstly a object of Simulation class needs to be initialized with the number of data bits, the static CIR and the number of iterations to average over. After the object has been initialized the internal function for the desired outcomes can be called as follows.

Table 1: Functions to call for desired simulations

| Desired Simulation | Function                                |
|--------------------|-----------------------------------------|
| DFE Static CIR     | <code>dfeStaticCIR(modulation)</code>   |
| DFE Dynamic CIR    | <code>dfeDynamicCIR(modulation)</code>  |
| MLSE Static CIR    | <code>MLSEStaticCIR(modulation)</code>  |
| MLSE Dynamic CIR   | <code>MLSEDynamicCIR(modulation)</code> |

Where the modulation parameter passed is in the chosen modulation type to simulate. Firstly the

functions generate data bits using the bit generator from practical 1(`generate_N_bits()`) and the data bit length set in the constructor of the simulation class. Secondly the functions converts the generated bits to a modulated signal using the `bits_to_symbols()` function from practical 1. In the case that a Dynamic CIR simulation is being run a random CIR will be generated using the `generateRandomCIR()`. The channel input response will then be passed into the `channelModification()` function this will add the channel response using the bellow equation.

$$r_t = s_t c_0 + s_{t-1} c_1 + s_{t-2} c_2 \quad (1.4)$$

To achieve the full channel response simulated by equation 1.5 below the modulated data that was received back from the `channelModification()` will be passed into the `add_noise()` function from practical 1. In doing so the full channel response is obtained for each symbol because the `add_noise()` function adds the  $\sigma \left( \frac{\kappa + j\kappa}{\sqrt{2}} \right)$ .

$$r_t = s_t c_0 + s_{t-1} c_1 + s_{t-2} c_2 + \sigma \left( \frac{\kappa + j\kappa}{\sqrt{2}} \right) \quad (1.5)$$

Using the above mentioned steps the sent symbols is calculated. The sent symbols is then passed into the chosen simulation DFE or MLSE. The MLSE or DFE then returns the optimal symbol to use. The detected data will then be demodulated using the `bit_detection()` from practical one. The BER is then determined by passing in the the generated Data bits and the demodulated received symbols. The BER is then added to an array. The above mentioned process will repeat the same number of times as specified by the number iterations. When the end of the iterations is reached the data is averaged and stored at the given SNR location.

The above mentioned method is repeated for the SNR in range of 0dB to 15dB with a pluse 0.1dB step and plotted with 151 equal spaced data points.



## 5 Results

The results of the simulation are given below

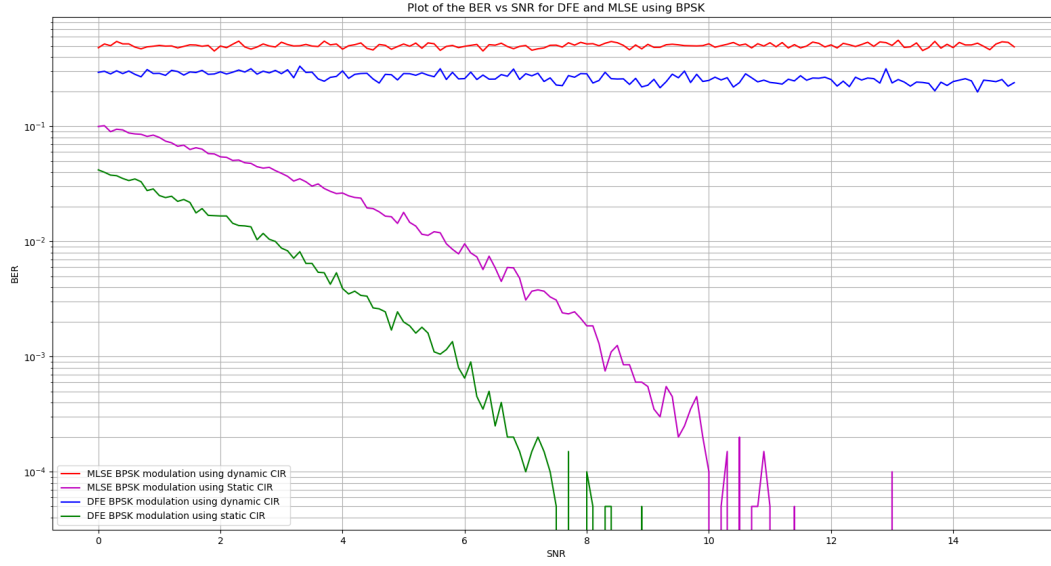


Figure 5.0.1: BER as a function of SNR for a BPSK with DFE and MLSE averaged over 100 iterations

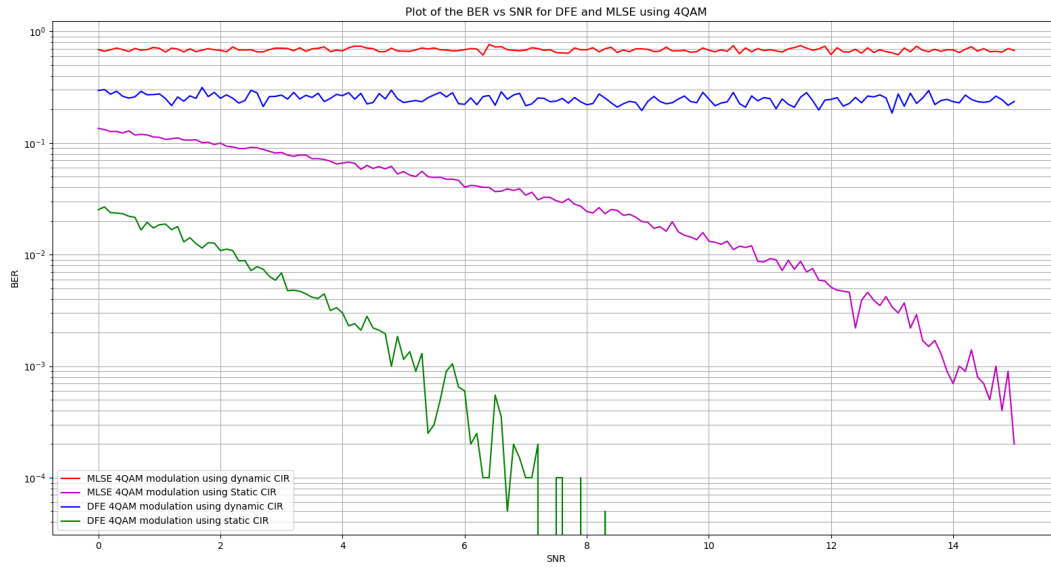


Figure 5.0.2: BER as a function of SNR for a 4QAM with DFE and MLSE averaged over 100 iterations

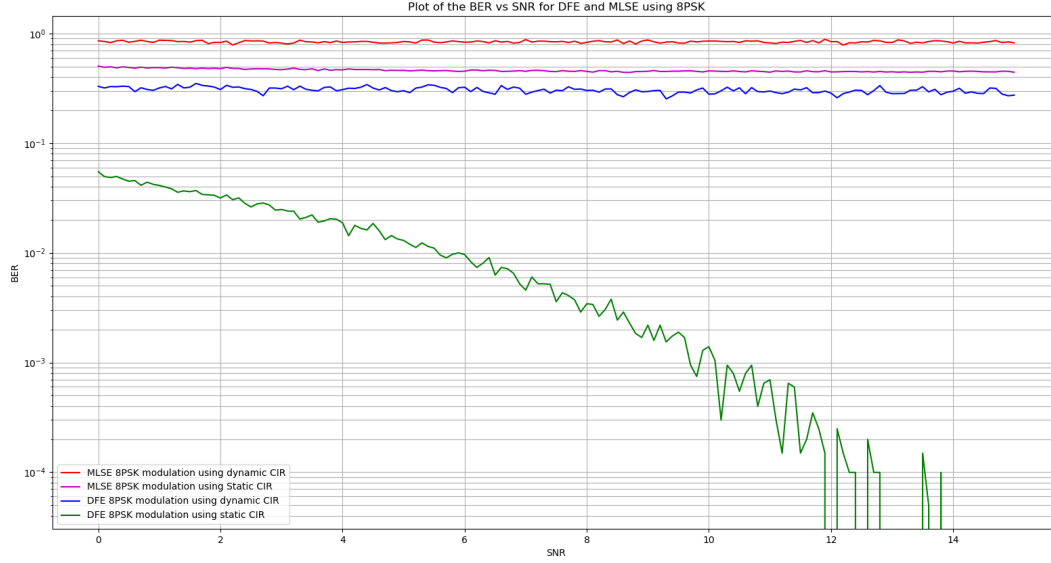


Figure 5.0.3: BER as a function of SNR for a 8PSK with DFE and MLSE averaged over 100 iterations

## 6 Discussion

The results in Figures 5.0.1 through till 5.0.3 on page 7 allow us to compare the performance of two different equalizers with different types of channel impulse responses and different modulation types. In reality, the impulse response of a channel is not always static. Randomness exists and thus needs to be simulated to gather realistic data. This is done by using a random gaussian number generator to generate a dynamic channel response. From Figures 5.0.1 through till 5.0.3, we can see that the static impulse response of the channels is much better than the dynamic impulse response of the channels. This is seen as an exponential decrease in the linearly decreasing BER of the channel impulse response compared to the slight decrease in the dynamic generated graphs of both the DFE and MLSE algorithms.

For a static channel impulse response, both algorithms for al constalation types exponentially decreases the BER to as the SNR increases and starts of with a lower BER when compared to the dynamic generated channel response. This is in line with our expectations that, as the the SNR increases, the distortions of the symbol sequence declines, leading to fewer errors in the symbols sent.

We can use the simulation results to compare the performance of 2 equalizers. We find that DFE outperforms MLSE in both static and dynamic channel impulse responses. The DFE algorithm, which uses a static channel impulse response, is slightly better than MLSE. The DFE algorithm's BER reduction rate is much higher than the MLSE. There are obvious differences between the algorithms when using a dynamic channel impulse response. Both remain relatively constant, and the DFE has a low BER over the entire range of the simulated SNR values.

From this we can conclude that the DFE is far more suitable for the dynamic CIR than the MLSE algorithm with a lower bit transfer.

## 7 Conclusion

In conclusion the Decision Feedback Equalizer performs better than the Maximum Likelihood Sequence Estimation when using both a static and dynamic channel input response. This is however not as expected but can be due the the small but arrays being transported. It is expected that the MLSE will outperform the DFE when larger amount of data is being transferred since the MLSE makes a decision on the entire state space, while the DFE algorithm on take the consideration of the current state space. Thus the DFE will be at an advantage when comparing a small amount of bits, because if it predicts the wrong symbol the effect would be much smaller since the array of bits is smaller.

In the real world, due to the influence of multipath, there is almost always a dynamic CIR. Thus from the simulations it can be concluded that DFE achieves a much better BER, when a small amount of bits is transferred. This can be seen by comparing the average BER for all constellation types. It can be seen that the DFE algorithm always has a lower BER than that of MLSE.

# Bibliography

- [1] M. S. John Proakis, *Digital Communications, 5th Edition*, 5th ed. McGraw-Hill Science Engineering Math, 2007. [Online]. Available: <http://gen.lib.rus.ec/book/index.php?md5=0d60b29a68495e82cdafb3dc7708ff0a>

# Appendix A

## DFE Code

```
1 import numpy as np
2 import math
3
4 #Function to return symbols of chosen
  constellation type
5 def BackSymbols(i,L):
6     #BPSK=1 #4QAM=2 8PSK=3
7     if i==1:
8         temp = []
9         temp.append(1)
10        temp.append(-1)
11        memory= [1]*(L-1) #the first 1 is
the memory symbols
12        return temp,memory
13    elif(i==2):
14        temp = []
15        temp.append(round(1/2**0.5,4) +
round(1/2**0.5,4)*1j)
16        temp.append(round(-1/2**0.5,4) +
round(1/2**0.5,4)*1j)
17        temp.append(round(-1/2**0.5,4) +
round(-1/2**0.5,4)*1j)
18        temp.append(round(1/2**0.5,4) +
round(-1/2**0.5,4)*1j)
19        memory=[round(1/2**0.5,4) + round
(1/2**0.5,4)*1j]*(L-1)
20        return temp,memory
21    elif(i==3):
22        temp = []
23        temp.append(1)
24        temp.append(round(1/2**0.5,4) +
round(1/2**0.5,4)*1j)
25        temp.append(1j)
26        temp.append(round(-1/2**0.5,4) +
round(1/2**0.5,4)*1j)
27        temp.append(-1)
28        temp.append(round(-1/2**0.5,4) +
round(-1/2**0.5,4)*1j)
29        temp.append(-1j)
30        temp.append(round(1/2**0.5,4) +
round(-1/2**0.5,4)*1j)
31        memory=[round(1/2**0.5,4) + round
(1/2**0.5,4)*1j]*(L-1)
32        return temp,memory
33
34 #Function to Return Symbols after going
  through DFE
35 def DFE(recieved,channels,L ,modulation):
36
37     if(modulation == 'bpsk'):
38         temp = BackSymbols(1,L)
39         Options = list(temp[0])
40         symbols = list(temp[1])
41     elif(modulation == '4qam'):
42         temp = BackSymbols(2,L)
43         Options = list(temp[0])
44         symbols = list(temp[1])
45     elif(modulation == '8psk'):
46         temp = BackSymbols(3,L)
47         Options = list(temp[0])
48         symbols = list(temp[1])
49
50     a = 0 #Variable to iterate through all
symbols in recieved
51     for i in range(0,len(recieved)):
52         Symbols_temp=[]
53         n=len(channels)-1# length of the
channel L-1
54         #calculating the product but from
second position
55         sumof=0
56         for j in range(1,n):
57             sumof += symbols[n-1+a]*channels
[j]
58             n-=1
59
60         for k in Options:
61             Symbols_temp.append(np.abs(
recieved[i]-((k)*channels[0]+sumof))**2)
62             #Deltas is computed here
63             estimate=Options[Symbols_temp.index(
min(Symbols_temp))] # Minimum Delta is
retrieved
64             symbols.append(estimate)
65             a += 1
66     return symbols[L-1:] #final
```

# Appendix B

## MLSE Code

```

1 import math
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from question_1 import uniform
5 from question_3 import q3
6
7 def randuni(Data_length):
8     rand = []
9     for i in range(0, Data_length):
10         rand.append(uniform())
11     return rand
12
13 Data_length = 200
14
15 # The following three functions map the
16 # various symbols to bits
17 def BPSKmap(b):
18     _bpsk_symbols = []
19     for _sy in b:
20         if _sy == 1:
21             _bpsk_symbols.append(1)
22         else:
23             _bpsk_symbols.append(-1)
24     return _bpsk_symbols
25
26 def _4QAMmap(b):
27     _4QAM = []
28     M = 2
29     List_SUB = [b[n:n + M] for n in range(0,
30 len(b), M)]
31     for _sy in List_SUB:
32         if _sy == [0, 0]:
33             _4QAM.append(complex(1 / np.sqrt(
34 2), 1 / np.sqrt(2)))
35         elif _sy == [0, 1]:
36             _4QAM.append(complex(-1 / np.
37 sqrt(2), 1 / np.sqrt(2)))
38         elif _sy == [1, 1]:
39             _4QAM.append(complex(-1 / np.
40 sqrt(2), -1 / np.sqrt(2)))
41         elif _sy == [1, 0]:
42             _4QAM.append(complex(1 / np.sqrt
43 (2), -1 / np.sqrt(2)))
44
45     return _4QAM
46
47 def _8PSKmap(b):
48     _8PSK = []
49     M = 3
50     List_SUB = [b[n:n + M] for n in range(0,
51 len(b), M)]
52     for _sy in List_SUB:
53         if _sy == [0, 0, 0]:
54             _8PSK.append(complex(1, 0))
55         elif _sy == [0, 0, 1]:
56             _8PSK.append((1 + 1j) / np.sqrt
57 (2))
58         elif _sy == [0, 1, 1]:
59             _8PSK.append(1j)
60         elif _sy == [0, 1, 0]:
61             _8PSK.append((-1 + 1j) / np.sqrt
62 (2))
63         elif _sy == [1, 1, 0]:
64             _8PSK.append(-1)
65         elif _sy == [1, 1, 1]:
66             _8PSK.append((-1 - 1j) / np.sqrt
67 (2))
68         elif _sy == [1, 0, 1]:
69             _8PSK.append(-1j)
70         elif _sy == [1, 0, 0]:
71             _8PSK.append((1 - 1j) / np.sqrt
72 (2))
73     return _8PSK
74
75 # This function is used to add white noise
76 # to the signals
77 def noiseAddition(trans, _c_, L, M, snr):
78     r_symbols = []
79     p = snr
80     std_dev = 1 / np.sqrt(math.pow(10, (p /
81 10)) * 2 * math.log2(M))
82     q = q3(200)
83     for i in range(L - 1, len(trans)):
84         _s = q.gauss()
85         r_symbols.append(trans[i] * _c_[0] +
86 trans[i - 1] * _c_[0 + 1]
87 + trans[i - 2] * _c_
88 [2] + std_dev * (sample + (_s) * 1j))
89     return r_symbols # without the padding
90
91 # Function to calculate the bit error
92 def ERR_bits(sent, received_symbols):

```

```

82     error = 0
83     for k in range(0, len(received_symbols)):
84         if sent[k] != received_symbols[k]:
85             error += 1
86     BER = error / len(received_symbols) * 100
87     return BER
88
89 sqrt2 = np.sqrt(2)
90 c = [0.89 + 0.92j, 0.42 - 0.37j, 0.19 + 0.12j]
91 N = 200
92
93
94 # Function to reverse the sequence to the
95 # correct order since path is
96 # determined from the tail
97 def rev(x):
98     return x[::-1]
99
100 # The following three functions are used to
101 # generate deltas between nodes depending
102 # on the modulation scheme
103 def BPSKDelta(received_symbols, Symbols):
104     d = []
105     for s in Symbols:
106         for j in range(0):
107             d.append(np.abs(received_symbols[
108                 (j * c[0] + s[1] * c[2]) ** 2)
109             ])
110
111 def _4QAMDelta(received_symbols, Symbols):
112     d = []
113     for s in Symbols:
114         for j in range(0):
115             d.append(np.abs(received_symbols[
116                 (j * c[0] + s[1] * c[1] + s[0] * c[2]) ** 2)
117             ])
118
119 def _8PSKDelta(received_symbols, Symbols):
120     d = []
121     for s in Symbols:
122         for j in range(0):
123             d.append(np.abs(received_symbols[
124                 (j * c[0] + s[1] * c[1] + s[0] * c[2]) ** 2)
125             ])
126
127 # This function enables the MLSE algorithm
128 # to function as a medium with memory
129 def memoryGeneration(Length, index):
130
131     if index == 1:
132         0 = [1, -1]
133         m = [1] * (Length - 1)
134         return 0, m
135     elif (index == 2):
136         0 = [(1 + 1j) / np.sqrt(2), (-1 + 1j) / np.sqrt(2), (-1 - 1j) / np.sqrt(2), (1 - 1j) / np.sqrt(2)]
137         m = [(1 + 1j) / np.sqrt(2)] * (Length - 1)
138         return 0, m
139     elif (index == 3):
140         0 = [1, (1 + 1j) / np.sqrt(2), 1j, (-1 + 1j) / np.sqrt(2), -1, (-1 - 1j) / np.sqrt(2), -1j, (1 - 1j) / np.sqrt(2)]
141         m = [(1 + 1j) / np.sqrt(2)] * (Length - 1)
142         return 0, m
143
144 # The following three functions are the
145 # actual implementation of the
146 # MLSE functions for the different
147 # modulation schemes
148 def MLSE_BPSK(received_symbols, N):
149     b = []
150     for i in range(8):
151         b.append(format(8 + i, 'b'))
152     Symbols = []
153     for i in range(8):
154         Sym = []
155         for j in range(3):
156             if b[i][j + 1] == "1":
157                 Sym.append(1)
158             else:
159                 Sym.append(-1)
160         Symbols.append(Sym)
161     deltas = []
162     for i in range(N):
163         deltas.append(BPSKDelta(
164             received_symbols, Symbols, i))
165
166     trans = []
167     for i in range(N):
168         cost = min(deltas[N - 1 - i])
169         bit = deltas[N - 1 - i].index(cost)
170
171         if bit % 2 == 0:
172             trans.append(-1)
173         else:
174             trans.append(1)
175     trans = rev(trans)
176     return trans
177
178 def MLSE_4QAM(received_symbols, N):
179     b = []
180     for i in range(64):
181         b.append(format(64 + i, 'b'))
182     Symbols = []
183     i = 0
184     while i < 64:
185         Sym = []
186         for j in range(6):
187             if b[i][j + 1:j + 3] == "00":
188                 Sym.append((1 + 1j) / sqrt2)
189             elif b[i][j + 1:j + 3] == "01":

```

```

186         Sym.append((-1 + 1j) / sqrt2)
187     elif b[i][j + 1:j + 3] == "11":
188         Sym.append((-1 - 1j) / sqrt2)
189     elif b[i][j + 1:j + 3] == "10":
190         Sym.append((1 - 1j) / sqrt2)
191     Symbols.append(Sym)
192     i += 2
193
194     deltas = []
195     for i in range(N):
196         deltas.append(_4QAMDelta(
197             received_symbols, Symbols, i))
198
199     trans = []
200     for i in range(N):
201         cost = min(deltas[N - 1 - i])
202         bit = deltas[N - 1 - i].index(cost)
203
204         if bit % 4 == 0: # received_symbols
205             (1+1j)/sqrt2
206             trans.append((1 + 1j) / sqrt2)
207         elif bit % 4 == 1: #
208             received_symbols (-1+1j)/sqrt2
209             trans.append((-1 + 1j) / sqrt2)
210         elif bit % 4 == 2: #
211             received_symbols (-1-1j)/sqrt2
212             trans.append((-1 - 1j) / sqrt2)
213         elif bit % 4 == 3: #
214             received_symbols (1-1j)/sqrt2
215             trans.append((1 - 1j) / sqrt2)
216
217     trans = rev(trans)
218
219     return trans
220
221 def MLSE_8PSK(received_symbols=[], N=4):
222     L = 3
223     b = []
224     S = []
225     for i in range(512):
226         b.append(format(512 + i, 'b'))
227     Syms = []
228     i = 0
229     while i < 8 ** L:
230         S = []
231         for j in range(9):
232             if b[i][j + 1:j + 4] == "111":
233                 S.append(1)
234             elif b[i][j + 1:j + 4] == "110":
235                 S.append((1 + 1j) / sqrt2)
236             elif b[i][j + 1:j + 4] == "010":
237                 S.append(1j)
238             elif b[i][j + 1:j + 4] == "011":
239                 S.append((-1 + 1j) / sqrt2)
240             elif b[i][j + 1:j + 4] == "001":
241                 S.append(-1)
242             elif b[i][j + 1:j + 4] == "000":
243                 S.append((-1 - 1j) / sqrt2)
244             elif b[i][j + 1:j + 4] == "100":
245                 S.append(-1j)
246             elif b[i][j + 1:j + 4] == "101":
247                 S.append((1 - 1j) / sqrt2)
248         Syms.append(S)
249
250         i += 3
251
252     deltas = []
253     for i in range(N):
254         deltas.append(_8PSKDelta(
255             received_symbols, Syms, i))
256
257     trans = []
258     for i in range(N):
259         cost = min(deltas[N - 1 - i])
260         bit = deltas[N - 1 - i].index(cost)
261
262         if bit % 8 == 0:
263             trans.append(1)
264         elif bit % 8 == 1:
265             trans.append((1 + 1j) / sqrt2)
266         elif bit % 8 == 2:
267             trans.append(1j)
268         elif bit % 8 == 3:
269             trans.append((-1 + 1j) / sqrt2)
270         elif bit % 8 == 4:
271             trans.append(-1)
272         elif bit % 8 == 5:
273             trans.append((-1 - 1j) / sqrt2)
274         elif bit % 8 == 6:
275             trans.append(-1j)
276         elif bit % 8 == 7:
277             trans.append((1 - 1j) / sqrt2)
278
279     trans = rev(trans)
280     return trans
281
282 # Function to generate the values to plot
283 # the BER vs SNR for a static CIR
284 # taking the number of iterations as a
285 # parameter to find the average BER
286 def static(numIt, mod):
287     Data_length = N
288     q = q3(200)
289     bits = list(q.generate_N_bits())
290     BPSK_bits = BPSKmap(bits)
291     _4QAM_bits = _4QAMmap(bits)
292     EBPSK_bits = _8PSKmap(bits)
293     _c_ = [0.89 + 0.92j, 0.42 - 0.37j, 0.19
294           + 0.12j]
295     yB = []
296     y4 = []
297     y8 = []
298
299     # bpsk
300     if mod == 'b':
301         L = 3
302         M = 2
303         a, trans = memoryGeneration(3, 1)
304         trans.extend(BPSK_bits)
305         snr = 0
306         while (snr < 15):
307             y = []
308             for i in range(numIt):
309                 received_symbols =
310                 noiseAddition(trans, _c_, L, M, snr)
311                 DET = MLSE_BPSK(
312                 received_symbols, Data_length)
313                 y.append(ERR_bits(trans[L -
314                 1:], DET))
315             yB.append(sum(y) / len(y))

```



```

304         snr += 0.1
305
306     # 4Qam
307     if mod == '4':
308         L = 3
309         M = 4
310         a, trans = memoryGeneration(3, 2)
311         trans.extend(_4QAM_bits)
312         snr = 0
313         while (snr < 15):
314             y = []
315             for i in range(numIt):
316                 received_symbols =
noiseAddition(trans, _c_, L, M, snr)
317                 DET = MLSE_4QAM(
received_symbols, int(Data_length / 2))
318                 y.append(ERR_bits(trans[L -
1:], DET))
319                 y4.append(ERR_bits(trans[L -
1:], DET))
320             snr += 0.1
321
322     # 8psk
323     if mod == '8':
324         L = 3
325         M = 8
326         a, trans = memoryGeneration(3, 3)
327         trans.extend(EBPSK_bits)
328
329         snr = 0
330         while (snr < 15):
331             y = []
332             for i in range(numIt):
333                 received_symbols =
noiseAddition(trans, _c_, L, M, snr)
334                 DET = MLSE_8PSK(
received_symbols, int(Data_length / 3))
335                 y.append(ERR_bits(trans[L -
1:], DET))
336                 y8.append(ERR_bits(trans[L -
1:], DET))
337             snr += 0.1
338         if mod == 'b':
339             return yB
340         if mod == '4':
341             return y4
342         if mod == '8':
343             return y8
344
345 # Function to generate dynamic CIR
346 def generateRandomCIR(values=3):
347     c = []
348     q = q3(200)
349     for x in range(0, values):
350         temp = q.gauss()
351         temp = complex(temp, temp)
352         temp = temp/math.sqrt(2.3)
353         c.append(temp)
354     return c
355
356 # Function to generate the values to plot
the BER vs SNR for a dynamic
357 # CIR taking the number of iterations as a
parameter to find the average BER
358 def dynamic(numIt, mod):
359     Data_length = N

```

```

360     q = q3(200)
361     bits = list(q.generate_N_bits())
362     BPSK_bits = BPSKmap(bits)
363     _4QAM_bits = _4QAMmap(bits)
364     EBPSK_bits = _8PSKmap(bits)
365     yB = []
366     y4 = []
367     y8 = []
368     # bpsk
369     if mod == 'b':
370         L = 3
371         M = 2
372         a, trans = memoryGeneration(3, 1)
373         trans.extend(BPSK_bits)
374         snr = 0
375         while (snr < 15):
376             y = []
377             for i in range(numIt):
378                 _c_ = generateRandomCIR()
379                 received_symbols =
noiseAddition(trans, _c_, L, M, snr)
380                 DET = MLSE_BPSK(
received_symbols, Data_length)
381                 y.append(ERR_bits(trans[L -
1:], DET))
382                 yB.append(sum(y)/len(y))
383             snr += 0.1
384
385     # 4Qam
386     if mod == '4':
387         L = 3
388         M = 4
389         a, trans = memoryGeneration(3, 2)
390         trans.extend(_4QAM_bits)
391         snr = 0
392         while (snr < 15):
393             y = []
394             for i in range(numIt):
395                 _c_ = generateRandomCIR()
396                 received_symbols =
noiseAddition(trans, _c_, L, M, snr)
397                 DET = MLSE_4QAM(
received_symbols, int(Data_length / 2))
398                 y.append(ERR_bits(trans[L -
1:], DET))
399                 y4.append(ERR_bits(trans[L -
1:], DET))
400             snr += 0.1
401
402     #8PSK
403     if mod == '8':
404         L = 3
405         M = 8
406         a, trans = memoryGeneration(3, 3)
407         trans.extend(EBPSK_bits)
408         snr = 0
409         while (snr < 15):
410             y = []
411             for i in range(numIt):
412                 _c_ = generateRandomCIR()
413                 received_symbols =
noiseAddition(trans, _c_, L, M, snr)
414                 DET = MLSE_8PSK(
received_symbols, int(Data_length / 3))
415                 y.append(ERR_bits(trans[L -
1:], DET))
416                 y8.append(ERR_bits(trans[L -

```

```
1:], DET))
416         snr += 0.1
417     if mod == 'b':
418         return yB
419     if mod == '4':
420         return y4
421     if mod == '8':
422         return y8
```

# Appendix C

## Simulation Code

```
1 from math import sqrt
2 import numpy as np
3 from question_3 import q3
4 from MLSE import static
5 from MLSE import dynamic
6 import random
7 import copy
8 from DFE import DFE
9 import matplotlib.pyplot as plt
10
11 # Class to perform a simulation of the
12 # BER for the MLSE and DFE symbol estimation
13 # methods
14 class Simulation:
15     def __init__(self, n = 200, linC = [],
16                 numIterations = 20):
17         self.linC = linC
18         self.q3 = q3(n)
19         self.n = n
20         self.numIterations = numIterations
21
22     # Function to perform the MLSE
23     # simulation with dynamic CIR
24     # with a Gaussian random number
25     # generated from practical 1
26     def MLSEDynamicCIR(self, modulation):
27         print("Conducting a MLSE simulation", modulation, " with a Dynamic CIR")
28         if (modulation == "bpsk"):
29             BER = dynamic(self.numIterations, 'b')
30         elif(modulation == "4qam"):
31             BER = dynamic(numIterations, '4')
32         elif(modulation == "8psk"):
33             BER = dynamic(numIterations, '8')
34         return BER
35
36     # Function to perform the MLSE
37     # simulation with static CIR
38     # with a Gaussian random number
39     # generated from practical 1
40     def MLSEStaticCIR(self, modulation):
41         print("Conducting a MLSE simulation", modulation, " with a Static CIR")
42         if (modulation == "bpsk"):
43             BER = static(self.numIterations, 'b')
44         elif(modulation == "4qam"):
45             BER = static(numIterations, '4')
46         elif(modulation == "8psk"):
47             BER = static(numIterations, '8')
48         return BER
49
50     # Function to perform the DFE simulation
51     # with static CIR
52     # with a Gaussian random number
53     # generated from practical 1
54     def dfeStaticCIR(self, modulation):
55         numIter = self.numIterations
56         print("Conducting a DFE ", modulation, " simulation with a Static CIR")
57         BER = [] #store the average
58         SNR = 0
59         while (SNR < 15):
60             tempBER = [] #to store all the
61             # iterations
62             for count in range(0, numIter):
63                 myDataBits = self.q3.
64                 generate_N_bits()
65                 myDataBits = list(myDataBits)
66                 if(modulation == '8psk'):
67                     while(len(myDataBits) %
68                           3 != 0):
69                         myDataBits.append(1)
70
71                 modulatedSignal = self.q3.
72                 bits_to_symbols(myDataBits, modulation)
73                 modulatedSignal = list(
74                     modulatedSignal)
75
76                 convolutedSignal = self.
77                 channelModification(self.linC,
78                                     modulatedSignal, modulation)
79
80                 signalSent = self.q3.
81                 add_noise(convolutedSignal, SNR, self.q3
82                           .get_fbits(modulation))
83                 signalSent = list(signalSent
```

```

)
72                                     114
73         dataDetected = DFE(         115
signalSent, self.linC, 3, modulation)
74
75         demodulatedSignal = self.q3 116
bit_detection(dataDetected, modulation) 117
76         demodulatedSignal = list(    118
demodulatedSignal)                      119
77
78         tempBER.append(self.q3.       120
calculate_BER(myDataBits,              121
demodulatedSignal))                   122
79         del modulatedSignal          123
80         del convolutedSignal         124
81         del demodulatedSignal
82         BER.append((sum(tempBER)/len( 125
tempBER)))
83         SNR = SNR + 0.1              126
84         return BER                  127
85                                     128
86     # Function to perform the DFE simulation 129
with dynamic CIR                      130
87     # with a Gaussian random number      131
generated from practical 1            132
88     def dfeDynamicCIR(self, modulation): 133
89         numIter = self.numIterations    134
90         print("Conducting a DFE simulation " 135
,modulation ," with a Dynamic CIR")
91         BER = [] #store the average BER for
each SNR                             136
92         SNR = 0
93         while (SNR < 15):              137
94             tempBER = [] #to store all the
interations                          138
95             for count in range(0, numIter): 139
96
97                 myDataBits = self.q3.    140
generate_N_bits() #raw data bits      141
98                 myDataBits = list(myDataBits 142
)                                     143
99                 if(modulation == '8psk'): 144
100                     while(len(myDataBits) % 145
3 != 0):                             146
101                         myDataBits.append(1) 147
102                                     148
103                 modulatedSignal = self.q3. 149
bits_to_symbols(myDataBits, modulation) 150
104
105                                     151
106                 randomCIR = self.         152
generateRandomCIR()                  153
107                 convolutedSignal = self.  154
channelModification(randomCIR,        155
modulatedSignal, modulation)
108                 signalSent = self.q3.     156
add_noise(convolutedSignal, SNR, self.q3 157
.get_fbts(modulation))
109                 signalSent = list(signalSent 158
)                                     159
110
111                 dataDetected = DFE(       160
signalSent, randomCIR, 3, modulation) 161
112                 demodulatedSignal = self.q3 162
bit_detection(dataDetected, modulation)
113                 demodulatedSignal = list( 163
demodulatedSignal)

tempBER.append(self.q3.
calculate_BER(myDataBits,
demodulatedSignal))
del modulatedSignal
del convolutedSignal
del demodulatedSignal
BER.append((sum(tempBER)/len(
tempBER)))
SNR = SNR + 0.1
return BER

#returns a random CIR with "values"
elements
# uses the Gaussian random number
generator implemented in practical 1.
def generateRandomCIR(self, values=3):
c = []
for x in range(0,values):
temp = self.q3.gauss()
temp = complex(temp,temp)
temp = temp/sqrt(2.3)
c.append(temp)
temp = 0
return c

# function to add the effects of the
channel to the stream of symbols being
sent.
def channelModification(self, cir,
stream, modulation):
returnStream = list(copy.deepcopy(
stream))
helper = self.get_symbols(modulation
)
for x in range(0,len(stream)):
temp1 = 0 #sk-2
temp2 = 0 #sk-1
if(x-2 < 0):
temp1 = helper[0]
else:
temp1 = stream[x-2]
if(x-1 <0):
temp2 = helper[0]
else:
temp2 = stream[x-1]
returnStream[x] = (stream[x]*cir
[0] + temp2*cir[1] + temp1*cir[2])
return returnStream

def get_symbols(self, constalation = '
bpsk'):
temp = []
if (constalation == 'bpsk'):
temp.append(1)
temp.append(-1)
return temp #Returns BPSK symbol
list

elif(constalation == '4qam'):
temp.append(round(1/2**0.5,4) +
round(1/2**0.5,4)*1j)
temp.append(round(-1/2**0.5,4) +
round(1/2**0.5,4)*1j)
temp.append(round(-1/2**0.5,4) +

```

```

164         round(-1/2**0.5,4)*1j)
165         temp.append(round(1/2**0.5,4) + 208
166         round(-1/2**0.5,4)*1j)
167         return temp#Returns 4QAM symbol 209
168     list
169
170     elif(constalation == '8psk'):
171         temp.append(1)
172         temp.append(round(1/2**0.5,4) +
173         round(1/2**0.5,4)*1j)
174         temp.append(1j)
175         temp.append(round(-1/2**0.5,4) +
176         round(1/2**0.5,4)*1j)
177         temp.append(-1)
178         temp.append(round(-1/2**0.5,4) +
179         round(-1/2**0.5,4)*1j)
180         temp.append(-1j)
181         temp.append(round(1/2**0.5,4) +
182         round(-1/2**0.5,4)*1j)
183         return temp#Returns 8PSK symbol 215
184     list
185
186     # Function to perform the simulation of
187     all situations
188     def simulate(self):
189         #self.BPSKsim()
190         #self.QAMsim()
191         self.PSKsim()
192
193     def BPSKsim(self):
194         print("Plotting all")
195         SNR = np.linspace(0, 15, 151)
196         MLSEdynamic = self.MLSEdynamicCIR('
197         bpsk')
198         MLSEStatic = self.MLSEStaticCIR('
199         bpsk')
200         dfeRandom_bpsk = self.dfeDynamicCIR(
201         'bpsk')
202         dfeLin_bpsk = self.dfeStaticCIR('
203         bpsk')
204         plt.semilogy(SNR, MLSEdynamic, 'r-',
205         label='MLSE BPSK modulation using
206         dynamic CIR')
207         plt.semilogy(SNR, MLSEStatic, 'm-',
208         label='MLSE BPSK modulation using Static
209         CIR')
210         plt.semilogy(SNR, dfeRandom_bpsk, 'b
211         -', label='DFE BPSK modulation using
212         dynamic CIR')
213         plt.semilogy(SNR, dfeLin_bpsk, 'g-',
214         label='DFE BPSK modulation using static
215         CIR')
216         plt.grid(which='both')
217         plt.ylabel("BER")
218         plt.xlabel("SNR")
219         plt.title("Plot of the BER vs SNR
220         for DFE and MLSE using BPSK")
221         plt.legend(loc='best')
222         plt.show()
223         print("Complete")
224
225     def QAMsim(self):
226         print("Plotting all")
227         SNR = np.linspace(0, 15, 151)
228         MLSEdynamic = self.MLSEdynamicCIR('
229         4qam')
230         MLSEStatic = self.MLSEStaticCIR('
231         4qam')
232         dfeRandom_4qam = self.dfeDynamicCIR(
233         '4qam')
234         dfeLin_4qam = self.dfeStaticCIR('
235         4qam')
236         plt.semilogy(SNR, MLSEdynamic, 'r-',
237         label='MLSE 4QAM modulation using
238         dynamic CIR')
239         plt.semilogy(SNR, MLSEStatic, 'm-',
240         label='MLSE 4QAM modulation using Static
241         CIR')
242         plt.semilogy(SNR, dfeRandom_4qam, 'b
243         -', label='DFE 4QAM modulation using
244         dynamic CIR')
245         plt.semilogy(SNR, dfeLin_4qam, 'g-',
246         label='DFE 4QAM modulation using static
247         CIR')
248         plt.grid(which='both')
249         plt.ylabel("BER")
250         plt.xlabel("SNR")
251         plt.title("Plot of the BER vs SNR
252         for DFE and MLSE using 4QAM")
253         plt.legend(loc='best')
254         plt.show()
255         print("Complete")
256
257     def PSKsim(self):
258         print("Plotting all")
259         SNR = np.linspace(0, 15, 151)
260         MLSEdynamic = self.MLSEdynamicCIR('
261         8psk')
262         MLSEStatic = self.MLSEStaticCIR('
263         8psk')
264         dfeRandom_8psk = self.dfeDynamicCIR(
265         '8psk')
266         dfeLin_8psk = self.dfeStaticCIR('
267         8psk')
268         plt.semilogy(SNR, MLSEdynamic, 'r-',
269         label='MLSE 8PSK modulation using
270         dynamic CIR')
271         plt.semilogy(SNR, MLSEStatic, 'm-',
272         label='MLSE 8PSK modulation using Static
273         CIR')
274         plt.semilogy(SNR, dfeRandom_8psk, 'b
275         -', label='DFE 8PSK modulation using
276         dynamic CIR')
277         plt.semilogy(SNR, dfeLin_8psk, 'g-',
278         label='DFE 8PSK modulation using static
279         CIR')
280         plt.grid(which='both')
281         plt.ylabel("BER")
282         plt.xlabel("SNR")
283         plt.title("Plot of the BER vs SNR
284         for DFE and MLSE using 8PSK")
285         plt.legend(loc='best')
286         plt.show()
287         print("Complete")
288
289     # n = number of data bits (200)
290     # c = static CIR
291     # numIterations = number of iterations to
292     # take the average of, before plotting -
293     # adjust as needed.
294     # "mySim.simulate()" to run the simulation

```

```
        and retrieve the plot
246 # !!! Warning increasing numIteration
        improves tge accuracy of the graphs but
        takes much longer to execute !!!
247 n = 200
248 c = [complex(0.89,0.92), complex(0.42, -0.37)
        , complex(0.19,0.12)]
249 numIterations = 100
250 mySim = Simulation(n,c, numIterations)
251 mySim.simulate()
```

# Appendix D

## Question 3

```
1 import numpy as np
2 import random
3 import math
4 import time
5 import matplotlib.pyplot as plt
6
7 class q3:
8     def __init__(self, numBits):
9         self.numBits = numBits
10        self.s1 = 10
11        self.s2 = 20
12        self.s3 = 30
13
14        # Global seed variables
15
16
17        def seed_unifrom(self, seed1:int, seed2:int, seed3:int)->None:
18            """Advanced update of seed values
19            for URNG
20                :param seed1: the first seed value
21            for WH Algorithm
22                :param seed2: the second seed value
23            for WH Algorithm
24                :param seed3: the third seed value
25            for WH Algorithm
26            """
27            self.s1 = seed1
28            self.s2 = seed2
29            self.s3 = seed3
30        #end uniform()
31
32        def uniform(self)->float:
33            """Wichmann-Hill algorithm
34            Use global variables
35            """
36            # Write your Wichmann-Hill algorithm
37            code here
38            #Declaration of constant primes
39            constPrime1 = 40087
40            constPrime2 = 50423
41            constPrime3 = 61333
42
43            #Declaration of arbitrary
44            multipliers
45            constMul1 = 347
46
47            constMul2 = 346
48            constMul3 = 348
49
50            #Wichmann-Hill algorithm for
51            updating the seeds
52            self.s1 = (constMul1 * self.s1) %
53            constPrime1
54            self.s2 = (constMul2 * self.s2) %
55            constPrime2
56            self.s3 = (constMul3 * self.s3) %
57            constPrime3
58
59            #Restricting the result to [0,1)
60            result = self.s1 / (constPrime1 *
61            1.0) + self.s2 / (constPrime2 * 1.0) +
62            self.s3 /(constPrime3 * 1.0)
63            result = result - math.trunc(result)
64            return result
65        #end uniform()
66
67        global seed1
68        global seed2
69        seed1 = time.time()
70        seed2 = time.time()+2
71
72        def seed_gauss(self, s1:int, s2:int)->
73        None:
74            """Advanced update of seed values
75            for GRNG
76                :param seed1: the first seed value
77            for GH Algorithm
78                :param seed2: the second seed value
79            for GH Algorithm
80            """
81            global seed1
82            seed1 = s1
83            global seed2
84            seed2 = s2
85            return
86
87        def gauss(self)->float:
88            """Marsaglia-Bray algorithm
89            Use global variables
90            """
91            while True:
92                global seed1
93                global seed2
94                seed1 += 1
```

```

80         seed2 += 1
81         random.seed(seed1)
82         v1 = random.random() * 2.0 - 1.0
83         random.seed(seed2)
84         v2 = random.random() * 2.0 - 1.0
85
86         if ((v1 * v1 + v2 * v2 < 1.0)
and (v1 * v1 + v2 * v2 > 0.0)):
87             break
88
89         seed1 += 1
90         seed2 += 2
91         s = v1 * v1 + v2 * v2
92
93         x1 = v1 * math.sqrt(-2.0 * math.log(s) / s)
94         x2 = v2 * math.sqrt(-2.0 * math.log(s) / s)
95
96         return random.choice([x1,x2])
97
98     def generate_N_bits(self)->tuple:
99         # Use your uniform random number
100         generator to generate random bits. A
101         value larger
102         # than 0.5 is a 1 bit; a value less
103         than or equal to 0.5 is a 0 bit. This
104         will ensure a 50/50 0/1 bit ratio.
105         """
106         :param N: the number of values
107         to generate
108         :templist: temporary list to
109         store values and later convert to tuple
110         :return Tuple containing random
111         generated bits
112         """
113         templist = []
114         for i in range(self.numBits):
115             templist.append(int(np.round(
116 self.uniform()))))
117         return tuple(templist) #convert list
118         bit_stream into tuple and return
119         #end generate_N_bits()
120
121     def bits_to_symbols(self, bits:tuple,
122 constellations='bpsk')->tuple:
123         # Map the bits to symbols using the
124         respective modulation constellations
125         """
126         :param bits: tuple of bits to be
127         converted to symbols
128         :param constellations:
129         Constellation used for symbol conversion
130         :temp: temporary list to store
131         values and later convert to tuple
132         :return Tuple containing the
133         converted symbols
134         """
135         temp = []
136         message = ()
137         if(constellations == 'bpsk'):
138             for i in range(len(bits)):
139                 if(bits[i] == 1):#s1
140                     temp.append(1)
141                 elif(bits[i] == 0):#s2
142                     temp.append(-1)
143
144         #end of constellations == 'bpsk'
145
146         elif(constellations == '4qam'):
147             if(len(bits) % 2 == 0): #if the
148             number of is even
149                 i = 0
150                 while(i < len(bits) - 1):
151                     if(bits[i] == 0 and bits
152 [i + 1] == 0):
153                         temp.append(round
154 (1/2**0.5,4) + round(1/2**0.5,4)*1j) #s1
155
156                     elif(bits[i] == 0 and
157 bits[i + 1] == 1):
158                         temp.append(round
159 (-1/2**0.5,4) + round(1/2**0.5,4)*1j) #
160 s2
161
162                     elif(bits[i] == 1 and
163 bits[i + 1] == 1):
164                         temp.append(round
165 (-1/2**0.5,4) + round(-1/2**0.5,4)*1j) #
166 s3
167
168                     elif(bits[i] == 1 and
169 bits[i + 1] == 0):
170                         temp.append(round
171 (1/2**0.5,4) + round(-1/2**0.5,4)*1j) #
172 s4
173
174                 i += 2#post-increment
175                 index
176
177             #else: #for the case when there'
178             s an odd number of bits in the stream
179             #end of constellations == '4qam'
180
181         elif(constellations == '8psk'):
182             if(len(bits) % 3 == 0):
183                 i = 0
184                 while(i < len(bits) - 2):
185                     if(bits[i] == 1 and bits
186 [i + 1] == 1 and bits[i + 2] == 1): #s1
187                         temp.append(1)
188
189                     elif(bits[i] == 1 and
190 bits[i + 1] == 1 and bits[i + 2] == 0):
191                         #s2
192
193                         temp.append(round
194 (1/2**0.5,4) + round(1/2**0.5,4)*1j)
195
196                     elif(bits[i] == 0 and
197 bits[i + 1] == 1 and bits[i + 2] == 0):
198                         #s3
199
200                         temp.append(1j)
201
202                     elif(bits[i] == 0 and
203 bits[i + 1] == 1 and bits[i + 2] == 1):
204                         #s4
205
206                         temp.append(round
207 (-1/2**0.5,4) + round(1/2**0.5,4)*1j)
208
209                     elif(bits[i] == 0 and
210 bits[i + 1] == 0 and bits[i + 2] == 1):
211                         #s5
212
213                         temp.append(-1)

```



```

169                                     elif(bits[i] == 0 and
170 bits[i + 1] == 0 and bits[i + 2] == 0):
171                                     #s6
172                                     temp.append(round
173                                     (-1/2**0.5,4) + round(-1/2**0.5,4)*1j)
174                                     elif(bits[i] == 1 and
175 bits[i + 1] == 0 and bits[i + 2] == 0):
176                                     #s7
177                                     temp.append(-1j)
178                                     elif(bits[i] == 1 and
179 bits[i + 1] == 0 and bits[i + 2] == 1):
180                                     #s8
181                                     temp.append(round
182                                     (1/2**0.5,4) + round(-1/2**0.5,4)*1j)
183                                     i += 3 #post-increment
184                                     index
185                                     #else: #for the case when the
186                                     number of bits in the stream isn't a
187                                     multiple of 3
188                                     #end of constellations == '8psk'
189                                     elif(constellations == '16qam'):
190                                     if(len(bits) % 4 == 0):
191                                     i = 0
192                                     while(i < len(bits) - 3):
193                                     if(bits[i] == 1 and bits[i + 1] == 0 and bits[i + 2] == 0 and
194 bits[i + 3] == 0): #(1000)S1
195                                     temp.append(round
196                                     (1/2**0.5,4) + round(1/2**0.5,4)*1j)
197                                     elif(bits[i] == 1 and
198 bits[i + 1] == 0 and bits[i + 2] == 0
199 and bits[i + 3] == 1): #(1001)S2
200                                     temp.append(round
201                                     (1/2**0.5,4) + round(1/8**0.5,4)*1j)
202                                     elif(bits[i] == 1 and
203 bits[i + 1] == 1 and bits[i + 2] == 1
204 and bits[i + 3] == 1): #(1011)S3
205                                     temp.append(round
206                                     (1/2**0.5,4) + round(-1/8**0.5,4)*1j)
207                                     elif(bits[i] == 1 and
208 bits[i + 1] == 0 and bits[i + 2] == 1
209 and bits[i + 3] == 0): #(1010)S4
210                                     temp.append(round
211                                     (1/2**0.5,4) + round(-1/2**0.5,4)*1j)
212                                     elif(bits[i] == 1 and
213 bits[i + 1] == 1 and bits[i + 2] == 0
214 and bits[i + 3] == 0): #(1100)S5
215                                     temp.append(round
216                                     (1/8**0.5,4) + round(1/2**0.5,4)*1j)
217                                     elif(bits[i] == 1 and
218 bits[i + 1] == 1 and bits[i + 2] == 0
219 and bits[i + 3] == 1): #(1101)S6
220                                     temp.append(round
221                                     (1/8**0.5,4) + round(1/8**0.5,4)*1j)
222                                     elif(bits[i] == 1 and
223 bits[i + 1] == 1 and bits[i + 2] == 1
224 and bits[i + 3] == 1): #(1111)S7
225                                     temp.append(round
226                                     (1/8**0.5,4) + round(-1/8**0.5,4)*1j)
227                                     elif(bits[i] == 1 and
228 bits[i + 1] == 1 and bits[i + 2] == 1
229 and bits[i + 3] == 0): #(1110)S8
230                                     temp.append(round
231                                     (1/8**0.5,4) + round(-1/2**0.5,4)*1j)
232                                     elif(bits[i] == 0 and
233 bits[i + 1] == 1 and bits[i + 2] == 0
234 and bits[i + 3] == 0): #(0100)S9
235                                     temp.append(round
236                                     (-1/8**0.5,4) + round(1/2**0.5,4)*1j )
237                                     elif(bits[i] == 0 and
238 bits[i + 1] == 1 and bits[i + 2] == 0
239 and bits[i + 3] == 1): #(0101)S10
240                                     temp.append(round
241                                     (-1/8**0.5,4) + round(1/8**0.5,4)*1j)
242                                     elif(bits[i] == 0 and
243 bits[i + 1] == 1 and bits[i + 2] == 1
244 and bits[i + 3] == 1): #(0111)S11
245                                     temp.append(round
246                                     (-1/8**0.5,4) + round(-1/8**0.5,4)*1j)
247                                     elif(bits[i] == 0 and
248 bits[i + 1] == 1 and bits[i + 2] == 1
249 and bits[i + 3] == 0): #(0110)S12
250                                     temp.append(round
251                                     (-1/8**0.5,4) + round(-1/2**0.5,4)*1j)
252                                     elif(bits[i] == 0 and
253 bits[i + 1] == 0 and bits[i + 2] == 0
254 and bits[i + 3] == 0): #(0000)S13
255                                     temp.append(round
256                                     (-1/2**0.5,4) + round(1/2**0.5,4)*1j)
257                                     elif(bits[i] == 0 and
258 bits[i + 1] == 0 and bits[i + 2] == 0
259 and bits[i + 3] == 1): #(0001)S14
260                                     temp.append(round
261                                     (-1/2**0.5,4) + round(1/8**0.5,4)*1j)
262                                     elif(bits[i] == 0 and
263 bits[i + 1] == 0 and bits[i + 2] == 1
264 and bits[i + 3] == 1): #(0011)S15
265                                     temp.append(round
266                                     (-1/2**0.5,4) + round(-1/8**0.5,4)*1j)
267                                     elif(bits[i] == 0 and
268 bits[i + 1] == 0 and bits[i + 2] == 1
269 and bits[i + 3] == 0): #(0010)S16
270                                     temp.append(round
271                                     (-1/2**0.5,4) + round(-1/2**0.5,4)*1j)
272                                     i += 4
273                                     #else: #For the case when the
274                                     number of transmitted bits isn't a
275                                     multiple of 4
276                                     #end constellation == '16qam'
277                                     message = tuple(temp)

```

```

240         return message
241     #end bits_to_symbols()
242
243     def get_fbits(self, constellation='bpsk'):
244         """
245         This function returns the number
246         of bits per
247         symbol for a constellation.
248         """
249         if(constellation=='bpsk'):
250             return 1
251         elif(constellation == '4qam'):
252             return 2
253         elif(constellation == '8psk'):
254             return 3
255         elif(constellation == '16qam'):
256             return 4
257     #end get_fbits()
258
259     def sigma(self, snr:float, f_bits:int):
260         """
261         This function computes and
262         returns the sigma
263         value used in the computation of
264         symbol noise
265         based on the number of bits per
266         symbol in an
267         encoding scheme and the signal
268         to noise ratio.
269         """
270         return (1/math.sqrt((10**((snr/10))
271         *2*f_bits)))
272     #end sigma()
273
274     def add_noise(self, symbols:tuple, snr:
275     float, f_bits:int)->tuple:
276         """
277         This function computes and adds
278         noise to the
279         symbols tuple passed to it based
280         on the SNR and
281         f_bits passed in its parameters
282         """
283         temp = []
284         sigma_x = self.sigma(snr, f_bits)
285         for i in range(len(symbols)):
286             random_gaus = self.gauss()
287             to_add = complex(random_gaus,
288             random_gaus)
289             temp_val = symbols[i] + sigma_x
290             *(to_add/math.sqrt(2))
291             temp.append(temp_val)
292
293         return tuple(temp)
294     #end add_noise()
295
296     def euclidean_distance(self, symbol,
297     constellation='bpsk')->dict:
298         """
299         # return the distances between the
300         given symbol and every symbol in the
301         given constellation
302         # return your result as a dictionary
303         , with the symbol as the key to its
304         distance value
305         # eg {<symbol_1>: distance_1, ... ,
306
307     <symbol_n>: distance_n }
308     # >>> eculidean_distance( 0.34, '
309     bpsk')
310     # {1: 0.66 , -1: 1.34}
311     if(constellation=='bpsk'):
312         s1 = round(math.sqrt(abs(symbol
313         - 1)**2),4)
314         s2 = round(math.sqrt(abs(symbol
315         + 1)**2),4)
316         dictionary = {1:s1, -1:s2}
317         return dictionary
318     elif(constellation == '4qam'):
319         s1 = round(math.sqrt(abs(symbol
320         - complex(1/2**0.5, 1/2**0.5))**2),4)
321         s2 = round(math.sqrt(abs(symbol
322         - complex(-1/2**0.5, 1/2**0.5))**2),4)
323         s3 = round(math.sqrt(abs(symbol
324         - complex(-1/2**0.5, -1/2**0.5))**2),4)
325         s4 = round(math.sqrt(abs(symbol
326         - complex(1/2**0.5, -1/2**0.5))**2),4)
327
328         dictionary = {round(1/2**0.5,4)
329         + round(1/2**0.5,4)*1j: s1, round
330         (-1/2**0.5,4)
331         + round(1/2**0.5,4)*1j: s2,
332         round(-1/2**0.5,4) + round(-1/2**0.5,4)
333         *1j:s3,
334         round(1/2**0.5,4) + round
335         (-1/2**0.5,4)*1j:s4}
336         return dictionary
337     elif(constellation == '8psk'):
338         s1 = round(math.sqrt(abs(symbol
339         - 1)**2),2)
340         s2 = round(math.sqrt(abs(symbol
341         - complex(1/2**0.5, 1/2**0.5))**2),4)
342         s3 = round(math.sqrt(abs(symbol
343         - complex(0, 1))**2),2)
344         s4 = round(math.sqrt(abs(symbol
345         - complex(-1/2**0.5, 1/2**0.5))**2),4)
346         s5 = round(math.sqrt(abs(symbol
347         - (-1)**2),2)
348         s6 = round(math.sqrt(abs(symbol
349         - complex(-1/2**0.5, -1/2**0.5))**2),4)
350         s7 = round(math.sqrt(abs(symbol
351         - (complex(0, -1))**2),2)
352         s8 = round(math.sqrt(abs(symbol
353         - complex(1/2**0.5, -1/2**0.5))**2),4)
354
355         dictionary = {1:s1, round
356         (1/2**0.5,4) + round(1/2**0.5,4)*1j : s2
357         , 1j : s3,
358         round(-1/2**0.5,4) + round
359         (1/2**0.5,4)*1j : s4, -1 : s5, round
360         (-1/2**0.5,4) +
361         round(-1/2**0.5,4)*1j : s6, -1j:
362         s7, round(1/2**0.5,4) + round
363         (-1/2**0.5,4)*1j : s8}
364         return dictionary
365     elif(constellation == '16qam'):
366         s1 = round(math.sqrt(abs(symbol
367         - complex(1/2**0.5, 1/2**0.5))**2),4)
368         s2 = round(math.sqrt(abs(symbol
369         - complex(1/2**0.5, 1/8**0.5))**2),4)
370         s3 = round(math.sqrt(abs(symbol

```

```

- complex(1/2**0.5, -1/8**0.5)**2),4)
326     s4 = round(math.sqrt(abs(symbol 361
- complex(1/2**0.5, -1/2**0.5)**2),4) 362
327     s5 = round(math.sqrt(abs(symbol 363
- complex(1/8**0.5, 1/2**0.5)**2),4) 364
328     s6 = round(math.sqrt(abs(symbol 365
- complex(1/8**0.5, 1/8**0.5)**2),4) 366
329     s7 = round(math.sqrt(abs(symbol 367
- complex(1/8**0.5, -1/8**0.5)**2),4)
330     s8 = round(math.sqrt(abs(symbol
- complex(1/8**0.5, -1/2**0.5)**2),4) 368
331     s9 = round(math.sqrt(abs(symbol 369
- complex(-1/8**0.5, 1/2**0.5)**2),4) 370
332     s10 = round(math.sqrt(abs(symbol 371
- complex(-1/8**0.5, 1/8**0.5)**2),4) 372
333     s11 = round(math.sqrt(abs(symbol 373
- complex(-1/8**0.5, -1/8**0.5)**2),4) 374
334     s12 = round(math.sqrt(abs(symbol 375
- complex(-1/8**0.5, -1/2**0.5)**2),4) 376
335     s13 = round(math.sqrt(abs(symbol 377
- complex(-1/2**0.5, 1/2**0.5)**2),4) 378
336     s14 = round(math.sqrt(abs(symbol 379
- complex(-1/2**0.5, 1/8**0.5)**2),4) 380
337     s15 = round(math.sqrt(abs(symbol 381
- complex(-1/2**0.5, -1/8**0.5)**2),4)
338     s16 = round(math.sqrt(abs(symbol 382
- complex(-1/2**0.5, -1/2**0.5)**2),4)
339
340     dictionary = {round(1/2**0.5,4)
+ round(1/2**0.5,4)*1j : s1, 384
341     round(1/2**0.5,4) + round
(1/8**0.5,4)*1j : s2, 385
342     round(1/2**0.5,4) + round
(-1/8**0.5,4)*1j : s3, 386
343     round(1/2**0.5,4) + round
(-1/2**0.5,4)*1j : s4, 387
344     round(1/8**0.5,4) + round
(1/2**0.5,4)*1j : s5, 388
345     round(1/8**0.5,4) + round
(1/8**0.5,4)*1j : s6, 389
346     round(1/8**0.5,4) + round
(-1/8**0.5,4)*1j : s7, 390
347     round(1/8**0.5,4) + round
(-1/2**0.5,4)*1j : s8, 391
348     round(-1/8**0.5,4) + round
(1/2**0.5,4)*1j : s9, 392
349     round(-1/8**0.5,4) + round
(1/8**0.5,4)*1j : s10, 393
350     round(-1/8**0.5,4) + round
(-1/8**0.5,4)*1j : s11, 394
351     round(-1/8**0.5,4) + round
(-1/2**0.5,4)*1j : s12, 395
352     round(-1/2**0.5,4) + round
(1/2**0.5,4)*1j : s13, 396
353     round(-1/2**0.5,4) + round
(1/8**0.5,4)*1j : s14, 397
354     round(-1/2**0.5,4) + round
(-1/8**0.5,4)*1j : s15, 398
355     round(-1/2**0.5,4) + round
(-1/2**0.5,4)*1j : s16} 399
356     return dictionary 400
357 #end euclidean_distance() 401
358
359 def symbol_detection(self, symbols:tuple 402
, constellation='bpsk') -> tuple: 403
360     # return a tuple of the symbols 404

```

```

based on the euclidean_distances results
temp = []

for i in range(len(symbols)):
    temp_key = ''
    value = 999999999

    temp_dic = self.
    euclidean_distance(symbols[i],
    constellation)

    for key in temp_dic:
        if temp_dic[key] <= value:
            value = temp_dic[key]

    for key in temp_dic:
        if temp_dic[key] == value:
            temp.append(key)
            break

    return tuple(temp)
#end symbol_detection()

def bit_detection(self, symbols:tuple,
constellation='bpsk') -> tuple:
    """return a tuple of the bits based
    on the results of the symbols
    >>> bit_detection((1, -1, 1, -1), '
    bpsk')
    (1, 0, 1, 0)
    >>> bit_detection((1, 1, 1, -1), '
    bpsk')
    (1, 1, 1, 0)
    """
    temp = []
    tempsymbols = self.symbol_detection(
    symbols, constellation)
    tempsymbols = list(tempsymbols)
    if(constellation == 'bpsk'):
        for i in range(len(tempsymbols))
        :
            if(tempsymbols[i] == 1):
                temp.append(1)

            elif(tempsymbols[i] == -1):
                temp.append(0)
        #end of constellations == 'bpsk'

    elif(constellation == '4qam'):
        for i in range(len(tempsymbols))
        :
            if(tempsymbols[i] == round
            (1/2**0.5,4) + round(1/2**0.5,4)*1j):
                temp.append(0)
                temp.append(0)

            elif(tempsymbols[i] == round
            (-1/2**0.5,4) + round(1/2**0.5,4)*1j):
                temp.append(0)
                temp.append(1)

            elif(tempsymbols[i] == round
            (-1/2**0.5,4) + round(-1/2**0.5,4)*1j):
                temp.append(1)
                temp.append(1)

```

```

414         elif(tempsymbols[i] == round(466
(1/2**0.5,4) + round(-1/2**0.5,4)*1j): 467
415             temp.append(1) 468
416             temp.append(0) 469
417
418         else: 470
419             print("hier is groot
fout")
420
421
422     #end of constellations == '4qam' 473
423
424     elif(constellation == '8psk'): 475
425         for i in range(len(tempsymbols)): 476
:
426             if(tempsymbols[i] == 1):
427                 temp.append(1) 477
428                 temp.append(1) 478
429                 temp.append(1) 479
430
431             elif(tempsymbols[i] == round(481
(1/2**0.5,4) + round(1/2**0.5,4)*1j): # 482
s2
432                 temp.append(1)
433                 temp.append(1) 483
434                 temp.append(0) 484
435
436             elif(tempsymbols[i] == 1j): 485
#s3 486
437                 temp.append(0) 487
438                 temp.append(1) 488
439                 temp.append(0)
440
441             elif(tempsymbols[i] == round(489
(-1/2**0.5,4) + round(1/2**0.5,4)*1j): # 490
s4 491
442                 temp.append(0) 492
443                 temp.append(1) 493
444                 temp.append(1) 494
445
446             elif(tempsymbols[i] == -1): 495
#s5 496
447                 temp.append(0) 497
448                 temp.append(0) 498
449                 temp.append(1) 499
450
451             elif(tempsymbols[i] == round(500
(-1/2**0.5,4) + round(-1/2**0.5,4)*1j): #
#s6 501
452                 temp.append(0) 502
453                 temp.append(0) 503
454                 temp.append(0) 504
455
456             elif(tempsymbols[i] == -1j): 505
#s7 506
457                 temp.append(1)
458                 temp.append(0) 507
459                 temp.append(0) 508
460
461             elif(tempsymbols[i] == round(509
(1/2**0.5,4) + round(-1/2**0.5,4)*1j): # 510
s8 511
462                 temp.append(1) 512
463                 temp.append(0)
464                 temp.append(1) 513
465
#end of constellations == '8psk'
466
467     elif(constellation == '16qam'):
468         for i in range(len(tempsymbols))
:
469             if(tempsymbols[i] == round
(1/2**0.5,4) + round(1/2**0.5,4)*1j): #
(1000)S1
470                 temp.append(1)
471                 temp.append(0)
472                 temp.append(0)
473                 temp.append(0)
474
475             elif(tempsymbols[i] == round
(1/2**0.5,4) + round(1/8**0.5,4)*1j): #
(1001)S2
476                 temp.append(1)
477                 temp.append(0)
478                 temp.append(0)
479                 temp.append(1)
480
481             elif(tempsymbols[i] == round
(1/2**0.5,4) + round(-1/8**0.5,4)*1j): #
(1011)S3
482                 temp.append(1)
483                 temp.append(0)
484                 temp.append(1)
485                 temp.append(1)
486
487             elif(tempsymbols[i] == round
(1/2**0.5,4) + round(-1/2**0.5,4)*1j): #
(1010)S4
488                 temp.append(1)
489                 temp.append(0)
490                 temp.append(1)
491                 temp.append(0)
492
493             elif(tempsymbols[i] == round
(1/8**0.5,4) + round(1/2**0.5,4)*1j): #
(1100)S5
494                 temp.append(1)
495                 temp.append(1)
496                 temp.append(0)
497                 temp.append(0)
498
499             elif(tempsymbols[i] == round
(1/8**0.5,4) + round(1/8**0.5,4)*1j): #
(1101)S6
500                 temp.append(1)
501                 temp.append(1)
502                 temp.append(0)
503                 temp.append(1)
504
505             elif(tempsymbols[i] == round
(1/8**0.5,4) + round(-1/8**0.5,4)*1j): #
(1111)S7
506                 temp.append(1)
507                 temp.append(1)
508                 temp.append(1)
509                 temp.append(1)
510
511             elif(tempsymbols[i] == round
(1/8**0.5,4) + round(-1/2**0.5,4)*1j): #
(1110)S8
512                 temp.append(1)
513                 temp.append(1)
514

```

```

515         temp.append(1)
516         temp.append(0)
517
518         elif(tempsymbols[i] == round
(-1/8**0.5,4) + round(1/2**0.5,4)*1j ):
#(0100)S9
519             temp.append(0)
520             temp.append(1)
521             temp.append(0)
522             temp.append(0)
523
524         elif(tempsymbols[i] == round
(-1/8**0.5,4) + round(1/8**0.5,4)*1j): #
(0101)S10
525             temp.append(0)
526             temp.append(1)
527             temp.append(0)
528             temp.append(1)
529
530         elif(tempsymbols[i] == round
(-1/8**0.5,4) + round(-1/8**0.5,4)*1j):
#(0111)S11
531             temp.append(0)
532             temp.append(1)
533             temp.append(1)
534             temp.append(1)
535
536         elif(tempsymbols[i] == round
(-1/8**0.5,4) + round(-1/2**0.5,4)*1j):
#(0110)S12
537             temp.append(0)
538             temp.append(1)
539             temp.append(1)
540             temp.append(0)
541
542         elif(tempsymbols[i] == round
(-1/2**0.5,4) + round(1/2**0.5,4)*1j): #
(0000)S13
543             temp.append(0)
544             temp.append(0)
545             temp.append(0)
546             temp.append(0)
547
548         elif(tempsymbols[i] == round
(-1/2**0.5,4) + round(1/8**0.5,4)*1j): #
(0001)S14
549             temp.append(0)
550             temp.append(0)
551             temp.append(0)
552             temp.append(1)
553
554         elif(tempsymbols[i] == round
(-1/2**0.5,4) + round(-1/8**0.5,4)*1j):
#(0011)S15
555             temp.append(0)
556             temp.append(0)
557             temp.append(1)
558             temp.append(1)
559
560         elif(tempsymbols[i] == round
(-1/2**0.5,4) + round(-1/2**0.5,4)*1j):
#(0010)S16
561             temp.append(0)
562             temp.append(0)
563             temp.append(1)
564             temp.append(0)
565
566         return tuple(temp)
567     #end symbol_detection()
568
569     def calculate_SER(self, transmitted:
tuple, received:tuple)->float:
570         """
571         This function determines the
symbol error rate between the
transmitted and received
symbols passed into the function
as Tuples
572         """
573         counter = 0
574         for i in range(len(transmitted)):
575             if(transmitted[i] != received[i
]):
576                 counter += 1
577
578         total_transmitted = len(transmitted)
579         return counter/total_transmitted
580     #end calculate_SER()
581
582     def calculate_BER(self, transmitted:
tuple, received:tuple)->float:
583         """
584         This function determines the bit
error rate between the transmitted and
received
bits passed into the function as
Tuples
585         """
586         counter = 0
587         for i in range(len(transmitted)):
588             if(transmitted[i] != received[i
]):
589                 counter += 1
590
591         total_transmitted = len(transmitted)
592         return counter/total_transmitted
593     #end calculate_BER()
594
595     plt.show()

```

# Appendix E

## Question 1

```
1 import doctest
2 import numpy as np
3 import random
4 import math
5 import time
6
7
8 import matplotlib.pyplot as plt
9 import pandas as pd
10 import scipy.stats as stats
11
12 # declare global variables here, You may use
13     global variables in functions by using
14     the global key work
15 # Global seed variables
16 s1 = 10
17 s2 = 20
18 s3 = 30
19
20 def seed_uniform(seed1:int, seed2:int, seed3:
21     :int)->None:
22     """Advanced update of seed values for
23     URNG
24     :param seed1: the first seed value for
25     WH Algorithm
26     :param seed2: the second seed value for
27     WH Algorithm
28     :param seed3: the third seed value for
29     WH Algorithm
30     """
31     global s1, s2, s3
32     s1 = seed1
33     s2 = seed2
34     s3 = seed3
35 #end uniform()
36
37 def uniform()->float:
38     """Wichmann-Hill algorithm
39     Use global variables
40     """
41     # Write your Wichmann-Hill algorithm
42     code here
43     #Declaration of constant primes
44     global s1, s2, s3
45     constPrime1 = 40087
46     constPrime2 = 50423
47     constPrime3 = 61333
48
49     #Declaration of arbitrary multipliers
50     constMul1 = 347
51     constMul2 = 346
52     constMul3 = 348
53
54     #Wichmann-Hill algorithm for updating
55     the seeds
56     s1 = (constMul1 * s1) % constPrime1
57     s2 = (constMul2 * s2) % constPrime2
58     s3 = (constMul3 * s3) % constPrime3
59
60     #Restricting the result to [0,1)
61     result = s1 / (constPrime1 * 1.0) + s2 /
62         (constPrime2 * 1.0) + s3 / (constPrime3
63         * 1.0)
64     result = result - math.trunc(result)
65     return result
66 #end uniform()
67
68 def calculate_uniform_stats(N:int)->tuple:
69     """calculate the statistical parameters
70     for the random sequence give
71     :param N: the number of values to
72     generate
73     :return: tuple containing (mu, sigma,
74     sigma_2) in that order
75     """
76     values = [uniform() for x in range(N)]
77
78     mu = 0.0 #mean
79     sigma = 0.0 #standard deviation
80     sigma_2 = 0.0 # variance
81
82     #Computing the mean (mu)
83     for i in range(N):
84         mu += values[i]
85     mu = mu / (N * 1.0)
86
87     #Computing the variance (sigma^2)
88     for i in range(N):
89         sigma_2 += (values[i] - mu)**2
90     sigma_2 = sigma_2 / (N - 1)
91
92     #Computing the standard deviation (sigma)
93     sigma = (sigma_2)**0.5
94     return (mu, sigma, sigma_2)
```

```
82 #end calculate_uniform_stats()
```